

Behavioral Cloning for Self-Driving Cars: A Leak-Free Re-evaluation and a Negative Result on Temporal Models

Aly Elgemei
Siegen, Germany

<https://github.com/ali7996/behavioral-cloning-thesis>

2026

Abstract

Behavioral cloning trains a neural network to imitate a human driver by mapping a forward-facing camera image to a steering command. We revisit the practical component of a 2023 Master’s thesis on this task [Elgemei, 2023] and report a corrected, honestly-evaluated reproduction. Three issues are identified and addressed. First, the thesis’s two recurrent models were inadvertently trained on uniform random noise rather than on driving data; we retrain them on real ten-frame image sequences. Second, the thesis’s 2D-CNN+LSTM consumed a single frame reshaped into a length-one sequence and was therefore not a temporal model at all; we rebuild it as a genuine per-frame convolutional encoder followed by a recurrent head. Third, and most consequential, the conventional random train/validation split leaks information: consecutive frames are roughly 70 ms apart and nearly identical, so a random split places near-duplicates of validation frames into the training set. Re-evaluating the convolutional baseline on a leak-free contiguous split lowers its coefficient of determination from 0.52 to 0.31. On that leak-free split, neither temporal model beats the single-frame convolutional baseline; in fact neither beats predicting the mean steering angle ($R^2 \leq 0$). We report this negative result as-is: on this dataset, at this scale, temporal context does not improve steering prediction. All code, trained weights, and evaluation scripts are public.

1 Introduction

End-to-end behavioral cloning is the oldest and simplest paradigm for vision-based autonomous driving: a neural network is trained, by supervised regression, to reproduce the control output of a human demonstrator directly from raw camera input [Pomerleau, 1989, Bojarski et al., 2016]. It requires no hand-engineered perception stack and no explicit map, which makes it an attractive teaching and benchmarking task — and a frequent subject of student projects and theses.

This paper revisits one such project: the practical component of a 2023 Master’s thesis [Elgemei, 2023] that implemented and compared three architectures for predicting the steering angle of a car in the Udacity driving simulator. While re-examining the original code in order to publish it, we found methodological flaws that invalidated part of the thesis’s reported results. Rather than quietly discard the affected numbers, we treat the re-examination itself as the contribution: we correct the flaws, re-run the experiments under a leak-free evaluation protocol, and report the outcome — including a negative result — transparently.

Contributions.

1. We reproduce the convolutional baseline on the real driving dataset and find that, retrained honestly, it roughly halves the steering error reported in the thesis.
2. We document that the thesis’s two recurrent models were trained on `numpy` uniform-random arrays rather than on driving data, so their reported metrics measured nothing, and we retrain them on real ten-frame sequences.
3. We show that the conventional random frame split leaks information between training and validation, because adjacent frames in a continuous drive are near-duplicates; we introduce a contiguous, timeline-ordered split that removes the leak, and quantify its effect.
4. Under the leak-free protocol we report a negative result: temporal (recurrent) models do not improve on the single-frame convolutional baseline, and do not beat a constant mean predictor.

The intent of this report is not to claim a new architecture or a new state-of-the-art. It is to demonstrate that careful, honest evaluation of a small and familiar task can still produce a useful and non-obvious finding.

2 Background and Related Work

End-to-end driving. The idea of learning to steer directly from pixels dates to ALVINN, which trained a small fully-connected network to drive a vehicle from camera and range input [Pomerleau, 1989]. The approach was revived at scale by NVIDIA’s DAVE-2 system, in which a convolutional network maps a single front-facing camera frame to a steering command and learns useful road features without explicit supervision for them [Bojarski et al., 2016]. The architecture used in this paper is the DAVE-2 network, and the dataset is generated in a driving simulator of the same family.

Behavioral cloning and its limits. Plain behavioral cloning is known to be brittle. Codevilla et al. [2018] show that an unconditioned policy cannot resolve the ambiguity of an intersection — the same image admits a left or a right turn — and condition the policy on a navigational command to fix this. Codevilla et al. [2019] further document that behavior cloning suffers from high variance across training seeds, sensitivity to dataset composition, and a driving-performance ceiling that more data does not lift. Our negative result on temporal models is consistent with this literature: behavior cloning’s difficulties are not obviously solved by adding model capacity or input context.

Recurrent models for sequences. Long Short-Term Memory networks [Hochreiter and Schmidhuber, 1997] are the standard tool for modelling temporal dependencies, and it is natural to ask whether a short window of consecutive frames lets a driving model exploit motion that a single frame cannot. The thesis posed exactly this question. We show below that answering it requires both real sequential data and a split that does not leak temporally-adjacent frames — neither of which the original implementation had.

Leakage in sequential data. When samples are drawn from a continuous process, examples that are close in time are statistically dependent. A uniformly random train/validation split then places near-identical examples on both sides of the partition, and the validation score measures memorization of almost-seen inputs rather than generalization. This is a well-understood pitfall for time-series and video data; we quantify its size for the present task in Section 5.2.

3 Task, Dataset, and Preprocessing

Task. Given a forward-facing camera image (or a short window of consecutive images), the model predicts a scalar steering angle in $[-1, 1]$. This is a regression problem, and we evaluate it with regression metrics. We deliberately drop the pseudo-accuracy $1/(1 + \text{MSE})$ used in the original thesis: it is a monotone transform of the error, not a meaningful “accuracy”, and it makes weak models look strong.

Dataset. We use the public Udacity simulator dataset `rslim087a/track`: 4,053 frames recorded on a single circuit, with center, left, and right camera images and synchronized telemetry. Inspecting the timestamps encoded in the frame filenames, we find the recording is one uninterrupted run at roughly 14 Hz — the largest gap between consecutive frames is 0.48 s, and there are no gaps above 0.5 s. This single-run structure is what makes the leakage analysis of Section 5.2 both necessary and clean.

Steering balancing. The raw steering distribution is dominated by near-zero (straight-ahead) values. We bin the steering range into 25 bins and cap each bin at 400 samples, which reduces the dataset to 1,463 frames and prevents the model from collapsing onto the constant zero prediction.

Side cameras. Each frame carries three cameras. We use all three: the left and right images are added with a fixed steering correction of ± 0.15 , which both triples the data (to 4,389 image/steering pairs) and teaches recovery toward the lane center.

Preprocessing. Every image is cropped to remove sky and car hood, converted from RGB to YUV, blurred with a 3×3 Gaussian kernel, resized to 200×66 , and scaled to $[0, 1]$. During training only, four augmentations are each applied independently with probability 0.5: random zoom, pan, brightness change, and horizontal flip (with a sign flip of the steering label).

4 Models

We study three architectures. All are trained with the Adam optimizer [Kingma and Ba, 2015] ($\text{lr} = 10^{-3}$) and a mean-squared-error loss.

Convolutional baseline (single frame). This is the DAVE-2 network [Bojarski et al., 2016]: five convolutional layers (filter counts 24–36–48–64–64) followed by three fully-connected layers and a scalar output, with ELU activations and dropout. It has 252,219 trainable parameters and consumes one preprocessed $66 \times 200 \times 3$ frame.

Flatten-LSTM (ten frames). This is the thesis’s recurrent design: a window of ten consecutive frames, each flattened to a 39,600-dimensional vector, processed by two stacked LSTM [Hochreiter and Schmidhuber, 1997] layers of 128 units, then a dense head. Flattening the image discards all spatial structure and makes the first recurrent layer enormous; the model has 20,506,113 parameters. We retain this design as-is so that the temporal experiment is a faithful correction of the thesis rather than a redesign.

CNN-LSTM (ten frames). The thesis’s third model was described as a 2D-CNN+LSTM, but its implementation applied the convolutional stack to a *single* frame and then reshaped the result into a sequence of length one before the LSTM — so the recurrent layers saw no temporal axis and the model was not temporal at all. We rebuild it correctly: a strided convolutional encoder is applied per frame (time-distributed) across the ten-frame window, producing a genuine length-ten feature sequence, which an LSTM then summarizes. The corrected model has 1,105,701 parameters.

5 Two Corrections to the Evaluation

5.1 Training on real sequences

In the original code, the two recurrent models were trained not on driving data but on an array of uniform random noise — in effect, a call to `numpy.random.random` producing a tensor of shape $4389 \times 10 \times 66 \times 200 \times 3$ — paired with the real steering labels. A network trained on random inputs cannot learn an input-output mapping, so the LSTM and 2D-CNN+LSTM metrics reported in the thesis do not measure learning from driving data. Notably, those metrics were *better* than the convolutional baseline’s, which alone should have been a warning sign.

We correct this by building real temporal windows. Frames are sorted by their timestamp, split into maximal runs with no inter-frame gap above 0.5 s (here, a single run), and a window of ten consecutive frames is slid along each run. Each window is labelled with the steering angle of its final frame. Windows are built independently for the three cameras, so a window is always single-camera and remains a coherent short video. Augmentation is drawn once per window and applied identically to all ten frames, so temporal coherence is preserved.

5.2 A leak-free contiguous split

At 14 Hz, consecutive frames are about 70 ms apart and visually almost identical. A uniformly random train/validation split therefore places, for almost every validation frame, one or two near-duplicate frames into the training set. The model can score well on validation by recognizing inputs it has effectively already seen.

This matters for any of the three models, but it is fatal for the temporal experiment: ten-frame windows sampled one step apart overlap in nine of ten frames, so a random split over windows leaks almost completely.

We therefore order all frames by time and split the *timeline*: the first 80% of the run becomes the training region and the last 20% the validation region. Windows are built within each region only, so no window crosses the boundary and no validation window shares a frame with a training window. For the temporal models we balance the training windows by steering bin *after* windowing (balancing before windowing would punch holes in the timeline). For a fair comparison, the convolutional baseline is then re-scored on this same contiguous validation set, by predicting on the final frame of every validation window.

6 Experiments

Setup. All models are trained on a single CPU. The convolutional baseline is trained for 10 epochs. The temporal models are trained with early stopping on the validation loss; both stopped within a few epochs. Reported metrics are mean squared error (MSE), mean absolute error (MAE), root mean squared error (RMSE), and the coefficient of determination R^2 . An R^2 of 0 corresponds to

predicting the mean steering angle of the validation set; a negative R^2 is worse than that constant baseline.

6.1 Convolutional baseline vs. the thesis

Trained on the real data with the original preprocessing and augmentation intact, and evaluated under the thesis’s own protocol (a random 80/20 split), the convolutional baseline roughly halves the error the thesis reported (Table 1).

Table 1: Convolutional baseline under the thesis’s own random-split protocol.

	MSE	MAE	RMSE	R^2
This work (real data, random split)	0.054	0.168	0.233	0.520
Thesis [Elgemei, 2023], reported	—	0.300	0.408	—

6.2 Leak-free comparison of all three models

Table 2 reports all three models on the identical leak-free contiguous validation set (2,406 windows). The result is unambiguous: the single-frame convolutional baseline is the only model that explains any steering variance at all. Both temporal models have a negative R^2 — they are beaten by predicting the mean steering angle — despite one of them having eighty times the baseline’s parameters.

Table 2: All three models on the same leak-free contiguous validation set (2,406 windows). Lower is better for MSE/MAE/RMSE; higher is better for R^2 .

Model	Parameters	MSE	MAE	RMSE	R^2
Convolutional baseline (1 frame)	252,219	0.019	0.106	0.138	+0.308
Flatten-LSTM (10 frames)	20,506,113	0.029	0.128	0.169	−0.030
CNN-LSTM (10 frames)	1,105,701	0.029	0.138	0.170	−0.047

6.3 The size of the leakage effect

Table 3 isolates the leakage by evaluating the *same* trained convolutional model under the two splits. Moving from the random split to the leak-free contiguous split lowers R^2 from 0.52 to 0.31: about two-fifths of the apparent explained variance under the random split was an artifact of near-duplicate frames crossing the partition. (The MAE figures are not comparable across the two rows, because the random and contiguous validation sets have different steering distributions; R^2 , being normalized by each set’s own variance, is the meaningful cross-split quantity.)

Table 3: The same convolutional model evaluated under two splits. The random split inflates the apparent fit.

Split	Validation frames	RMSE	R^2
Random 80/20	800	0.233	0.520
Contiguous (leak-free)	2,406	0.138	0.308

7 Discussion

Why the temporal models fail. The failure is not a bug in the corrected pipeline; it is a genuine property of the task at this scale. After balancing, only about 2,300 ten-frame windows remain, and because windows are slid one step at a time they overlap heavily — the number of effectively independent short clips is far smaller still. Against this, the flatten-LSTM carries twenty million parameters. Its training loss never fell below roughly 0.18 MSE: the model did not even fit the training set well, so it is underfitting, not merely overfitting. The plausible explanation is that on this single, fixed circuit the correct steering angle is almost fully determined by the appearance of the current frame; a short window of recent frames adds little that the current frame does not already contain, and the recurrent models pay for the extra input dimensionality without a corresponding signal to learn from.

A negative result is still a result. The honest conclusion is that, for this dataset and this scale, temporal context does not help, and the simple convolutional baseline should be preferred. This contradicts the original thesis, whose recurrent models appeared to win — but only because they were evaluated on noise. Reporting the corrected outcome, rather than the flattering one, is the point of this paper.

Random splits flatter sequential models. The leakage measurement is a transferable warning. Any project that draws training examples from a continuous recording — driving logs, video, sensor streams — and then splits them uniformly at random will over-report generalization. A timeline-ordered split costs nothing and removes the illusion.

Limitations. The study uses a single simulated circuit, one human demonstrator, and a few thousand frames. A larger and more varied corpus — multiple tracks, real driving, harder maneuvers — could change the balance: temporal context may well help when there is enough data to learn it and enough situational variety to require it. The negative result is a statement about this regime, not a universal claim. Evaluation is also offline; closed-loop driving in the simulator is supported by the released code but is not scored here.

8 Conclusion

We revisited a Master’s-thesis behavioral-cloning project, found that its recurrent models had been trained on random noise and that its evaluation split leaked temporally-adjacent frames, and corrected both. Retrained honestly, the convolutional baseline roughly halves the thesis’s reported error; evaluated without leakage, it is the only one of the three models that explains any steering variance, while the two temporal models fail to beat a constant predictor. The contribution is not a new method but a demonstration that disciplined, leak-free evaluation of a small and familiar task yields a clear, reportable, and initially counter-intuitive finding. All code, trained weights, and evaluation scripts are available at <https://github.com/ali7996/behavioral-cloning-thesis>.

Reproducibility. The repository contains the data pipeline, all three model definitions, training and evaluation scripts, the contiguous-split sequence builder, and the saved metric files from which every number in this paper is taken. The dataset is the public `rslim087a/track` collection.

References

- Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Praseem Goyal, Lawrence D. Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, Xin Zhang, Jake Zhao, and Karol Zieba. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.
- Felipe Codevilla, Matthias Müller, Antonio López, Vladlen Koltun, and Alexey Dosovitskiy. End-to-end driving via conditional imitation learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- Felipe Codevilla, Eder Santana, Antonio M. López, and Adrien Gaidon. Exploring the limitations of behavior cloning for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- Aly Elgemei. Artificial neural networks implementation in self-driving car behavioral cloning. Master’s thesis, 2023.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Dean A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1989.